CareerSphere: Explainable Skill-Gap Planning for Campus Placement with Opportunity Math, Exact Workshop Planning and a Tool-Grounded LLM Copilot

V. Venkata Rupin, E. Manideep Reddy, J. Pruthvi Sai, S. Jaffer Ali and [Guide name]
Department of Computer Science and Engineering, [College name], [City], India
[email1]@[college].edu, [email2]@[college].edu

***Abstract*—Campus placement platforms usually report a match score and a list of missing skills, but they do not tell a student which skill to learn first, and they give the placement cell no principled way to choose training for the whole batch. We present CareerSphere, a placement platform for students, recruiters and placement officers built on a transparent match score: the weighted share of a job's required skills a student has, where must-have skills count twice as much as nice-to-have ones and a skill below the required level earns half credit, with a 70% "within reach" threshold and the recruiter's CGPA cutoff. Levels can be verified by server-graded assessments or by certificates checked by the placement cell. On top of it we define Opportunity Math, which ranks each missing skill by how many open jobs it would bring within reach, and a Workshop Planner that selects k skill workshops for the batch. Because skills pay off in combination, the planning objective is not submodular and one-at-a-time greedy selection can fail, so the planner uses exact search over skill combinations with bit-mask scoring and invites each student only to the workshops they need. Two large language model (LLM) features are grounded in the same data: a Career Copilot that answers only through eight read-only tools via Gemini function calling, and a resume skill finder that keeps an AI suggestion only if its quoted evidence occurs in the resume. On a fictional batch of 35 students and 12 jobs, Opportunity Math unlocks 1.43 jobs per student against 1.03 for recommending the most-demanded missing skill, and a Skill Roadmap that searches every bundle of up to three learning steps raises this to 2.89 with two steps and 4.71 with three, helping all 7 students for whom no single skill helps. A reverse search, Hidden Talent, shows recruiters 10 consented students who fit a job without having applied. With three workshops, exact planning gives all 35 students a job within reach (138 matches) where greedy selection reaches 32 (76 matches), and the full plan is computed in 6.5 ms.**

***Index Terms*—campus placement, job matching, explainable recommendation, skill gap analysis, combinatorial optimization, large language models, function calling.**

I. Introduction

Campus recruitment in Indian engineering colleges is a high-volume, short-window process. Students apply to many openings with little idea of which ones they are close to, recruiters screen applicants by hand, and the placement cell plans skill training largely on intuition. Much of the published work on this problem either predicts whether a student will be placed [3] or recommends jobs with learned models [1], [2]. Such outputs describe a situation; they rarely say what to do about it.

We take the view that a placement score is useful only if it points to an action. A student's question is "which one skill opens the most doors for me?", and the placement cell's question is "which k workshops should we run, and for whom, to help the most students?". Both can be answered exactly, and explained line by line, if the underlying match score is transparent. This paper makes four contributions:

- A transparent, level-aware match model and **Opportunity Math**, a counterfactual ranking of skills to learn or level up by the number of open jobs each would bring within reach, used consistently for students, recruiters and the placement cell, extended by a **Skill Roadmap** that finds the best bundles of up to three steps exactly, and by **Hidden Talent**, the same rule run from the job's side.
- A formulation of **batch workshop planning** as maximizing students reached (or matches created) with k workshops. We show with a minimal example why the objective is not submodular, and give an exact bit-mask search that is fast for practical k, with minimal per-student invitations.
- Two **grounded LLM features**: a function-calling Copilot restricted to read-only tools whose actions the student must confirm, and resume skill extraction whose suggestions are kept only when their quoted evidence is found in the resume.
- An implementation (Spring Boot, React, MySQL, with Mac, iPhone and Android clients) with 56 automated tests, evaluated on a fictional batch against demand-based heuristics and greedy selection.

Section II reviews related work, Section III describes the system, Section IV the methods, Sections V and VI the experiments and results, and Section VII the limitations.

II. Related Work

A. Job Recommendation and Person–Job Fit

De Ruijt and Bhulai [1] review a decade of job recommender systems and note the importance of temporal and reciprocal aspects, as well as the limited attention paid to fairness. Zhu et al. [2] learn joint representations of resumes and job postings for person–job fit; such models can be accurate but need large labelled histories and offer little explanation of an individual score. Placement prediction studies, for example Alfana et al. [3], estimate the probability that a student is placed from academic and cognitive features. They predict an outcome but do not identify which change would improve it.

B. Explainable Recommendation

Zhang and Chen [4] survey explainable recommendation and distinguish post-hoc explanations from models that are interpretable by construction. CareerSphere is of the second kind: every score is computed from the listed skills, so the explanation (matched and missing skills, CGPA rule) is the computation itself rather than an approximation of it.

C. Coverage and Submodular Maximization

Choosing k items to cover as many users as possible is a maximum coverage problem. For monotone submodular objectives the greedy algorithm achieves a $1 - 1/e$ approximation [5], which is the best possible in polynomial time unless P = NP [6]; Krause and Golovin [7] survey the area. Our objective has a threshold (70% of a job's skills), which creates complementarities between skills and breaks submodularity, so the greedy guarantee does not apply (Section IV-C).

D. Tool Use and Grounding in Language Models

Toolformer [8] and ReAct [9] show that language models can decide when to call external tools and use the results in their answers; commercial APIs expose the same pattern as function calling [16]. Ji et al. [10] survey hallucination in natural language generation, and Gao et al. [11] show that even strong models often fail to support their statements with citations. We restrict the Copilot to read-only tools over the student's own data and verify extracted resume evidence by exact string matching, so that unsupported claims are dropped rather than shown.

E. Fairness in Algorithmic Hiring

Raghavan et al. [12] examine the claims of vendors of algorithmic pre-employment assessments and the practical limits of debiasing. CareerSphere uses no demographic attributes; its only inputs are skills and the recruiter's published CGPA cutoff, and every score can be audited by hand.

III. System Overview

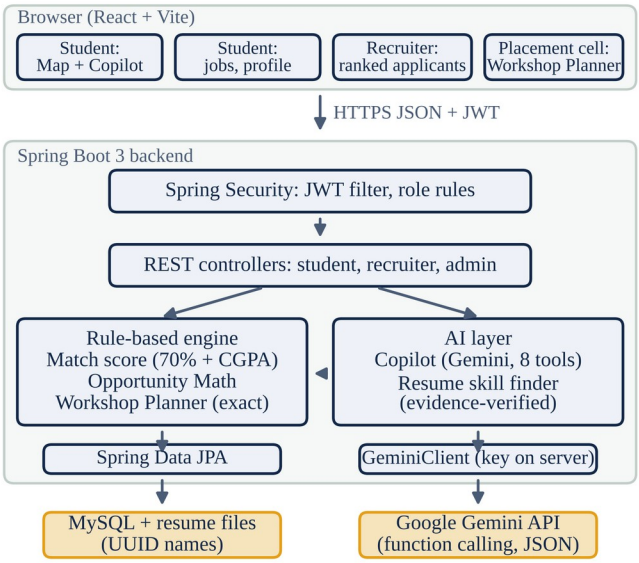


Fig. 1. Architecture. The rule-based engine computes every score; the AI layer reads the same data through tools and never writes on its own.

CareerSphere serves three roles (Fig. 1). **Students** maintain a profile and resume, see every open job placed on a Career Map, simulate learning a skill, and apply. **Recruiters** post jobs with required skills and an optional CGPA cutoff, and see applicants ranked by match score with an explanation. The **placement cell** sees batch analytics, including a demand–supply gap per skill, and uses the Workshop Planner.

A guided **placement profile** takes each student through four steps: upload a résumé, give placement consent (without which the student cannot apply and recruiters cannot see the profile), verify skills, and record a one-minute introduction video in the browser. A deterministic résumé score out of 100 is computed from ten fixed rules (for example 4 points per listed skill up to five, 5 per verified skill up to three), and every missing point is shown with how to earn it. A readiness leaderboard ranks the batch by the résumé score plus 15 points per verified skill, 10 per verified certificate and 5 per job within reach.

The frontend is a React application; the backend is a Spring Boot 3 service with Spring Data JPA over MySQL. Authentication uses stateless JSON Web Tokens [13] signed with HMAC and valid for eight hours; passwords are stored as BCrypt hashes [14]. Spring Security restricts each URL prefix to one role, and services check ownership, so a recruiter can see only applicants to their own jobs. A unique (job, student) database constraint prevents duplicate applications even under concurrent requests. Resume uploads are limited to PDF, DOC and DOCX up to 5 MB, checked against the file signature, and stored under random UUID names. The Gemini API key is held only by the server.

IV. Methods

A. Explainable Match Score

For a student s with skill set H_s, level $\ell_s(x) \in \{0, 1, 2, 3\}$ for each skill (0 = not known; 1–3 = Beginner, Intermediate, Advanced) and CGPA g_s, and an open job j with required skills R_j,

minimum levels $m_j(x)$, weights $w_j(x) = 2$ for a must-have and 1 for a nice-to-have skill, and optional cutoff c_j:

$$\text{score}(s, j) = \text{round}(100 \cdot \Sigma_x\, w_x \kappa_x \,/\, \Sigma_x\, w_x) \qquad (1)$$

with sums over $x \in R_j$, $w_x = w_j(x)$ and credit $\kappa_x = \kappa(\ell_s(x), m_j(x))$, where $\kappa(\ell, m) = 1$ if $\ell \geq m$, ½ if $0 < \ell < m$, and 0 if $\ell = 0$. Arithmetic is done in integer half-units, so the planner and the explanation shown to the student never disagree by rounding. When every skill is a must-have at Beginner level, (1) reduces to $\text{round}(100 \cdot |H_s \cap R_j| \,/\, |R_j|)$; the demo jobs use these defaults, so all results in Section VI also hold for the unweighted rule.

$$\text{reach}(s, j) = [\text{score}(s, j) \geq 70] \wedge [g_s \geq c_j] \qquad (2)$$

A job is *within reach* when (2) holds. With the defaults the threshold admits 3 of 4 and 4 of 5 required skills but not 2 of 3 (67%); a job needing Java at Intermediate, Spring Boot and Git scores 83% for a student with Java at Beginner and the other two skills. The CGPA rule is reported separately and never changes the score, so a student can see that a job is a good skill match but closed to them, and why.

B. Opportunity Math

An *action* is learning a skill x or raising it to a level some open job asks for. For each action the gain is the number of open jobs that would newly come within reach:

$$\text{gain}_s(x) = |\{\, j : \neg\text{reach}_H(s, j) \wedge \text{reach}_{H \cup x}(s, j) \,\}| \qquad (3)$$

where the subscript names the skill levels used in (2); for each skill the target level with the largest gain (then the lowest level) is kept, so the student sees either “Learn Docker” or “Level up Java to Advanced”. Actions are ranked by gain, with ties broken by the number of open jobs requiring the skill. A what-if simulator applies the same rule to any set of skills. On the Career Map (Fig. 2) each open job is drawn at distance 100 − score from the student, so the 70% ring separates jobs within reach from the rest, and toggling a skill “lever” moves jobs along visible trails.

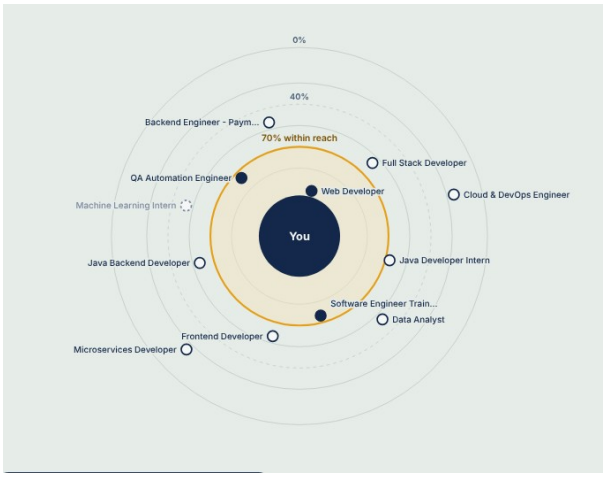


Fig. 2. Career Map for the demo student: 3 of 12 open jobs lie inside the 70% ring. Distance from the centre is 100 − match score; hollow points are below 70%, the dashed point fails a CGPA cutoff.

C. Workshop Planning

Let S be the students, J the open jobs and C the candidate skills (every skill some open job requires). A plan is a set $T \subseteq C$ with $|T| = k$; a workshop raises every attendee to at least Intermediate in its skill, so with default jobs a student effectively has $H_s \cup T$. Writing $n_s(T)$ for the number of jobs within reach of s after the plan, the two objectives are

$$f_{\text{reach}}(T) = |\{\, s : n_s(T) > 0 \,\}|, \quad f_{\text{match}}(T) = \Sigma_s\, n_s(T) \qquad (4)$$

The goal *Reach every student* maximizes $(f_{\text{reach}}, f_{\text{match}})$ lexicographically; *Most matches overall* maximizes $(f_{\text{match}}, f_{\text{reach}})$. The placement cell chooses the goal; the tool shows the trade-off rather than hiding it in a weight.

The objective is not submodular. Consider one student with skills {Git} and one job requiring {Java, Spring Boot, Git}. Then f({Java}) = f({Spring Boot}) = f(∅) = 0, because 2 of 3 skills is 67%, while f({Java, Spring Boot}) = 1. The marginal value of Spring Boot grows once Java is chosen, the opposite of diminishing returns. A greedy planner sees zero gain for each skill alone and may never choose either, so the 1 − 1/e guarantee of [5] does not hold. Section VI shows this happening on the demo batch.

Exact search. Candidate skills are indexed and every student and job becomes a bit mask, so testing (2) costs a few ORs, ANDs and population counts per machine word (one mask per level and weight class). The planner enumerates all $C(|C|, k)$ subsets, scores each in $O(|S| \cdot |J|)$, and keeps the best for the goal (Algorithm 1). If $C(|C|, k) \cdot |S| \cdot |J|$ exceeds 3×10^8, it falls back to greedy selection and says so on screen.

Invitations. For each student the planner finds the smallest subset of T that yields the same n_s as all of T (at most 2^k checks), and invites the student only to those workshops; a workshop that nobody needs is dropped. Workshops are then ordered by their immediate effect on the goal, and before and after totals are reported per step. The greedy plan is also computed and shown for comparison.

Algorithm 1: Exact workshop planning

input: students S, open jobs J, budget k, goal
$C \leftarrow$ skills required by some job in J; encode masks
if $C(|C|, k) \cdot |S| \cdot |J| > 3 \times 10^8$ **then return** greedy plan
best $\leftarrow \varnothing$
for each $T \subseteq C$ with $|T| = k$ **do**
  compute $f_{\text{reach}}(T)$, $f_{\text{match}}(T)$ by (2) and (4)
  if better for the goal than *best* **then** *best* $\leftarrow T$
for each $s \in S$ **do** $T_s \leftarrow$ smallest subset of *best* with the same n_s
drop skills in no T_s; order by immediate effect
return plan, invite lists T_s, totals, greedy comparison

D. Tool-Grounded Career Copilot

The Copilot uses Gemini function calling [16]. The model is given eight read-only tools: the student’s snapshot, job matches, skill opportunities (3), a learning simulation, a job explanation, application status, and two action proposals (apply to a job; show a skill on the map). The server executes each call for the student identified by the JWT, so the model cannot query another student’s data. The loop allows at most five tool rounds, and each model turn is sent back unchanged so that Gemini’s thought signatures are preserved. Proposed actions appear as buttons and run only when the student clicks them; every answer lists the data it used. If no API key is configured, or the API fails after retries and a fallback to a lighter model, a rule-

based offline mode answers the five most common questions with the same tools and says so.

E. Evidence-Verified Resume Skill Extraction

Text is extracted with Apache PDFBox or Apache POI. Gemini, in JSON mode, returns candidate skills, each with a quoted evidence line. The server normalizes case, whitespace and punctuation, and keeps a suggestion only if the quote occurs in the resume text and the skill maps to the catalog. An invented skill therefore cannot reach the student, however plausible it sounds. Without a key, word-boundary patterns with explicit disambiguation are used (JavaScript is not Java; "react quickly" is not React; a grade of C is not the C language). The student confirms each suggestion before it is added to the profile.

F. Verified Skill Levels

Self-reported levels are easy to inflate, so levels can be verified in two ways. A **skill assessment** draws 8 multiple-choice questions (4 basic, 4 advanced) for one of 15 skills, shuffles the options and stores the order with the attempt; the browser never receives the answers, and grading happens on the server within a 10-minute limit. Eight correct answers verify Advanced, six or seven Intermediate, and four or five Beginner; below four nothing is verified and a retake is possible after 24 hours. A verified level replaces the self-reported one in (1) and is shown to recruiters. A **certificate** from an outside course or exam (with issuer, credential ID or verification link and the file) is checked by the placement cell; when verified it adds its skill at Intermediate. Assessments are unlocked only after the résumé and placement consent, mirroring the order used by university placement portals.

G. Skill Roadmap and Hidden Talent

Opportunity Math looks one step ahead, but the example of Section IV-C applies to a single student as much as to a batch: a student with only Git gains nothing from Java or from Spring Boot alone, yet both together bring a Java, Spring Boot and Git job within reach. The **Skill Roadmap** therefore searches exactly over bundles B of 1, 2 and 3 actions, at most one per skill, drawn from the actions of Section IV-B, and for each size keeps the bundle that maximizes the number of jobs within reach, then minimizes the effort

$$\text{effort}(B) = \Sigma_{(*x*,\ \ell) \in *B*} (\ell - \ell_s(x)) \qquad (5)$$

that is, the number of levels gained. A bundle is shown only if it beats every smaller one, and it is marked *works together* when its gain exceeds the sum of its steps' single gains (3), which is exactly the non-submodular case. Steps are ordered by immediate effect, and for each nearby job the cheapest subset of that job's own gaps that brings it within reach is listed, or the CGPA cutoff is named when no skill can help. With about 30 candidate actions a student needs roughly 10^3 bundle checks, so the search runs on every page load.

Hidden Talent runs the rule from the job's side. For a recruiter's job it scores every student who gave placement consent and has not applied, and lists those within reach and those one action away, with the action named. The recruiter can send an invitation, which appears on the student's Jobs page; a unique (job, student) constraint prevents repeat invitations. Students without consent never appear, so the feature widens the pool without weakening the consent rule.

V. Experimental Setup

Data. We use the fictional demo batch shipped with the system: 35 students from four branches with 3.49 skills on average, and 12 open jobs from two fictional companies requiring 4.33 skills on average, 4 of them with a CGPA cutoff. There are 22 candidate skills and 420 student–job pairs. Scores are 0% for 144 pairs, 1–39% for 141, 40–69% for 91, 70–99% for 30 and 100% for 14. Today 26 students have at least one job within reach, with 44 matches in total.

Single-student baseline. We compare the top Opportunity Math skill with the *demand* heuristic: the missing skill required by the most open jobs, which is what a skill-gap list usually shows first. The metric is the number of jobs the recommended skill brings within reach.

Workshop baselines for k = 1 to 4: exact search (ours); *greedy*, which repeatedly picks the workshop with the largest immediate effect under the same goal and tie-breaking; *top-k demand*, the k skills required by the most open jobs; and *top-k gap*, the k skills with the largest difference between the share of jobs requiring a skill and the share of students who have it, as shown on the admin dashboard. For the two top-k baselines every student lacking a chosen skill attends. Runtime is the median of nine full planner calls after warm-up (Java 21 on a 2-core 2.8 GHz Intel Xeon).

VI. Results

A. Which Skill to Learn Next

TABLE I
Recommended Next Skill, 35 Students

Measure	Demand	Opportunity Math
Mean jobs brought within reach	1.03	**1.43**
Total jobs brought within reach	36	**50**
Students whose skill unlocks nothing	20	**7**
Students with a different recommendation	–	13

Opportunity Math is, by definition, the best single skill under (3); Table I quantifies how far the usual heuristic falls short. For 20 of 35 students, the most-demanded missing skill would not bring a single job within reach. For the remaining 7 students no single skill helps at all, because every job is two or more skills away, and the system says so instead of suggesting a skill that changes nothing.

TABLE IV
Skill Roadmap, 35 Students

Steps	Mean new jobs	Students helped
1 (Opportunity Math)	1.43	28
up to 2	2.89	**35**
up to 3	**4.71**	**35**

Table IV gives the Skill Roadmap results. All 7 students for whom no single skill helps gain at least one job with two steps; Rohan, for example, gains nothing from any one skill but three

jobs from Java and SQL together. Of the 65 multi-step bundles shown, 59 are marked *works together*, so on this batch the value of a second step usually comes from combining skills rather than adding independent gains. The search checks 1,083 bundles per student on average. Hidden Talent finds, over the 12 jobs, 10 consented students within reach of a job they have not applied to and 9 more one step away, against 14 applications actually made by those 10 students.

B. Planning Workshops for the Batch

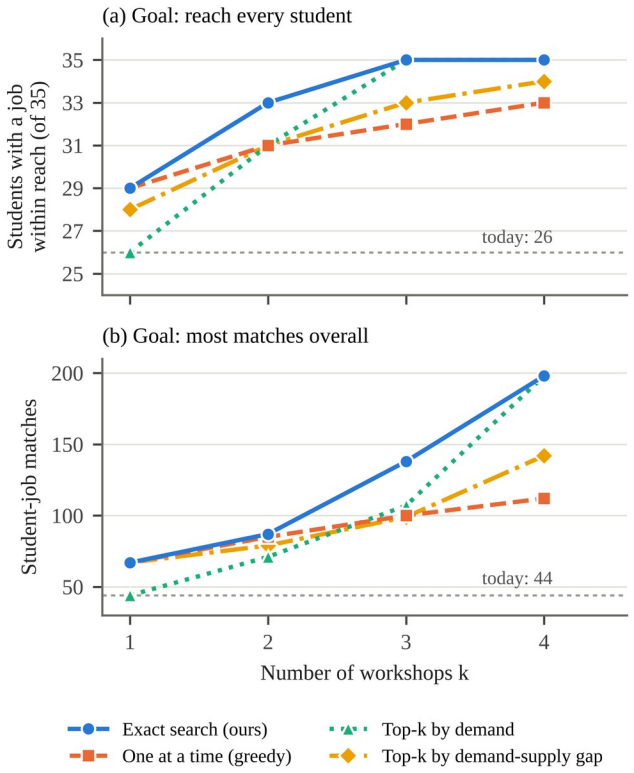


Fig. 3. Outcome of each planning method for $k = 1$ to 4 workshops on the demo batch (35 students, 12 open jobs). Dotted line: today. In (a) exact and greedy use the *Reach* goal (values in Table II); in (b) they use the *Matches* goal.

TABLE II
WORKSHOP PLANS: STUDENTS REACHED / MATCHES

k	Exact	Greedy	Top-k demand	Top-k gap
1	**29** / 56	29 / 56	26 / 44	28 / 67
2	**33** / 65	31 / 64	31 / 71	31 / 79
3	**35** / **138**	32 / 76	35 / 107	33 / 99
4	**35** / **198**	33 / 81	35 / **198**	34 / 142

Table II uses the *Reach every student* goal for exact and greedy plans. With three workshops, exact search selects Spring Boot, Java and Git and gives every student a job within reach, with 138 matches; greedy selects Docker, SQL and Git and reaches 32 students with 76 matches. Under the *Most matches overall* goal (Fig. 3b) greedy reaches 100 matches against 138. Greedy falls behind even the naive top-k demand baseline from $k = 3$, which illustrates Section IV-C: Java helps few students on its own and pays off only together with Spring Boot and Git, which one-at-a-time selection cannot see.

Exact search is never worse than any baseline on its own objective. On this batch the demand ranking coincides with the optimum at $k = 4$, which suggests demand is a reasonable prior but not a substitute for search: at $k = 2$ it reaches 31 students where the optimum (Data Structures and Java) reaches 33. At $k = 2$ the two goals also disagree: *Reach* chooses Data Structures and Java (33 students, 65 matches), while *Matches* chooses Spring Boot and Git (29 students, 87 matches). The three-workshop plan invites 31, 19 and 18 students, each only to the workshops they need.

TABLE III
PLANNER RUNTIME (MEDIAN OF 9 CALLS)

k	Combinations	Time (ms)
1	22	0.9
2	231	1.9
3	1,540	6.5
4	7,315	29.3
5	26,334	81.7
6	74,613	228.9

Table III shows that exact search is interactive for the workshop counts a placement cell plans for; times include the greedy comparison and the single-skill ranking computed in the same call.

C. Resume Skill Finder and Software Tests

On the demo resume, whose skills section lists Java, Python, JavaScript, SQL, HTML, CSS and Git, the finder reports six further skills found in project and achievement lines (Spring Boot, REST APIs, MySQL, React, Data Structures and Pandas), each with its quoted line. Adding them raises the student's jobs within reach from 3 to 8, showing that the profile undersold the student. The test suite has 39 unit tests (match rules with levels and nice-to-have weights, Opportunity Math, workshop planning including a complementary-skill case where greedy fails, roadmap bundles and Hidden Talent, the résumé score, server-side assessment grading, and resume patterns including false-positive guards) and 17 end-to-end API tests that run the application on an in-memory database through the real security filter chain, including placement consent, assessments, certificate review, the roadmap and invitations. Changing ≥ to > in the threshold rule made a unit test fail, confirming that the tests can detect a faulty rule.

VII. DISCUSSION AND LIMITATIONS

Data. All results are on a fictional batch designed to resemble a mid-sized department; they show the behaviour of the methods, not their effect on real placements. A human evaluation is prepared but not yet complete: 60 student–job pairs, stratified by score band, are labelled blind by three people, agreement is measured with Fleiss' κ [15], and the match rule is compared with keyword matching on accuracy, precision, recall and F1.

Model. Levels and must-have weights are modelled, but the weights (2 and 1) and the half credit are fixed design choices, and unverified levels are self-reported. The question bank (8 questions for each of 15 skills) is small enough to memorise

over retakes, so it supports a placement cell's screening rather than certification. The planner assumes that a workshop teaches its skill to every invitee and ignores capacity and cost, which matters when, for example, 31 of 35 students are invited to one workshop. Costs, capacities and partial learning can be expressed as an integer program, which would also scale beyond the point where exhaustive search is practical.

AI. On the free tier of the Gemini API, prompts may be used to improve the provider's products, so only demo data should be sent there; the Copilot's answers are limited to what its tools return, and the offline mode keeps the system usable when the API is unavailable. **Fairness.** The scores use no demographic attributes, and CGPA cutoffs are set and displayed by recruiters, but skill self-reporting can still favour students who describe themselves well, which the evidence-based resume finder only partly addresses [12].

VIII. Conclusion

CareerSphere turns a transparent match score into decisions: which skill a student should learn next, which applicants fit a job and why, and which workshops a placement cell should run, for whom. Treating workshop planning as a combinatorial problem, rather than a ranked list, matters because skills pay off together; on our demo batch, exact search reaches every student with three workshops where greedy selection leaves three behind, and a three-step roadmap more than triples what one well-chosen skill achieves for a student. Language models add value only where they are grounded in the same numbers, through read-only tools and verified evidence. Future work includes the human evaluation, a larger calibrated question bank, capacity-aware planning, and a study over a real placement season.

Acknowledgment

The authors thank [Guide name], [designation], Department of Computer Science and Engineering, [College name], for guidance throughout this project.

References

[1] C. de Ruijt and S. Bhulai, "Job recommender systems: A review," arXiv:2111.13576, 2021.

[2] C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li, "Person-job fit: Adapting the right talent for the right job with joint representation learning," *ACM Trans. Manage. Inf. Syst.*, vol. 9, no. 3, pp. 1–17, 2018.

[3] S. Alfana, S. K. R. Jammalamadaka, V. C. Prakash, and B. T. Reddy, "Cognitive and academic-based probability models for predicting campus placements," *IAES Int. J. Artif. Intell.*, vol. 11, no. 4, pp. 1239–1251, Dec. 2022, doi: 10.11591/ijai.v11.i4.pp1239-1251.

[4] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," *Found. Trends Inf. Retr.*, vol. 14, no. 1, pp. 1–101, 2020, doi: 10.1561/1500000066.

[5] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Math. Program.*, vol. 14, pp. 265–294, 1978.

[6] U. Feige, "A threshold of ln n for approximating set cover," *J. ACM*, vol. 45, no. 4, pp. 634–652, 1998.

[7] A. Krause and D. Golovin, "Submodular function maximization," in *Tractability: Practical Approaches to Hard Problems*, L. Bordeaux, Y. Hamadi, and P. Kohli, Eds. Cambridge, U.K.: Cambridge Univ. Press, 2014, pp. 71–104.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[11] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. EMNLP*, 2023, pp. 6465–6488, doi: 10.18653/v1/2023.emnlp-main.398.

[12] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT∗)*, 2020, pp. 469–481.

[13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[14] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999, pp. 81–91.

[15] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychol. Bull.*, vol. 76, no. 5, pp. 378–382, 1971.

[16] Google, "Function calling with the Gemini API," Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling (accessed Oct. 4, 2026).